# Docker, Compose & Swarm

**Date:** 2021-12-05 11:51:00

Table of contents:


- Overview
- Container CLIs
  Image CLIs
  - Tags
- Network CLIs
- General
- Docker Networks & DNS
  Dockerfile
  - Dockerfile Section Overview
- Persistent Data
  Compose
  - docker-compose.yml Structure
  - NGINX Example
  - Bigger example
  - Build example
  Swarm
  - Swarm Feature - Overlay Network
  - Swarm Feature - Routing Mesh
  - Service Example
  - Swarm Stacks
  - Inheritance, Secrets and Local Development
  - Swarm service updates
  - Labels & Constraints
- HEALTHCHECK
- Own Registry
- Cleanup
- Windows Containers
- Random Stuff
- References


### Overview

- docker container run
  - Manual cleanup needed (networks, volumes)
- docker-compose
  - Local development: Single node
    - interprets `build:` section in yml but not `deploy:`
  - Auto-Cleanup done on removal (networks, volumes)
  - "DNS Server" included so container can see each other and do not depend on IP
  - Can use `docker-compose.override.yml` for local dev and inheritance from file
- docker service
  - Swarm: Multi Node (manager & worker)
    - Support for secrets (via in-memory tmpfs)
    - Overlay network driver
    - Some Load-balancing features included via Routing Mesh
  - Manual Cleanup needed (networks, volumes, secrets)
  - service/task instead of container, as a service could have e.g. 3 replica
- docker stack
  - Swarm: Multi Node with compose file
    - minimum version of `3.1` required
    - interprets `deploy:` section in yml but not `build:`
    - `deploy` section supports for replicas, labels, placement constraints etc.
    - Support for secrets (via in-memory tmpfs)
      - via file (local development - supported via docker-compose
      - or external (production) - not supported via docker-compsoe
    - Overlay network driver
    - Some Load-balancing features included via Routing Mesh
  - Auto-Cleanup done on removal (networks, volumes, secrets)
- K8s
  - See K8s Blog Entry
- Container vs VM

### Container CLIs

| command | comment |
|---|---|
| docker verison | shows client/server version, client can talk to engine |
| docker info | config values for the engine |
| docker container run --publish 80:80 nginx | Start nginx container |
| docker container run --publish 80:80 --detach nginx | Run in Background |
| docker container run --publish 80:80 --name webhost nginx | Define container name |
| docker container run --publish 80:80 -it nginx /bin/bash | Interactive (-i)/pseudo tty (-t) |
| docker container start -ai <name/id> | Start a previously stopped container and connect to it attach (-a) / -i (interactive) |
| docker container ls -a --no-trunc | See command that was run for stopped containers |
| docker container exec -it <name/id> bash | Run an additional command in existing container* |
| docker container list [-a] | Show running containers [all=also stopped] |
| docker container stop <containerid/name> | stop a container that runs in background |
| docker stop <containerid/name> | Old style syntax to stop a container |
| docker container logs <id/name/first_digits_of_id> | shows container logs |
| docker container top <id> | shows processes running inside the container |
| docker container inspect <id> | show config details for contai. (volumes, network...) |
| docker container inspect --format "{{ .NetworkSettings.IPAddress }}" <id;id> | Uses GO template format, .=root |
| docker container inspect --format "{{ json .NetworkSettings }}" <id;id> \| jq | Using json output and JQ to format nicely on terminal |
| docker container stats <id> | shows stats for all (net I/O, block i/O cpu, mem) |
| docker container rm <id> | Remove container (-f for force=running container) |
| top -c -p $(pgrep -d',' -f mongo) | Verify resources that mongodb process/container is using |
| docker container run --publish 3306:3306 --env MYSQL_RANDOM_ROOT_PASSWORD=yes mysql | logs shows pw (-e == --env) |
| docker container port <id/name> | Check what ports are open & mapped for a container |
| docker container update <id/name> --memory <bytes> | set Memory in bytes of container |

- for example to connect to a container that runs mysql in detached mode
- `sudo -i` for interactive sudo (=get bash completion for docker!)

### Image CLIs

- Image = "Ordered Collection of root filesystem changes and corresponding execution parameters for use within a container runtime"
- Image = App binaries, dependencies & metadata about the image and how to run it
  - Not a complete OS, no kernle, no drivers, can be really small therefore
  - image consists of layers, each layer only stored once on host and has a unique SHA hash
  - Container = single r/w layer on of image
- Dockerhub
  - Official immages format <name> e.g. nginx whereas
  - Non-official images format <account/name> e.g. conrad/nginx
- Union File System
  - Basically presents all layers as a single fileystem (merges layer transparently)
  - Each layer has its own unqie SHA sum (caching)
    - For example two images that both use ubuntu as base, ubuntu only downloaded once
  - A container adds a writeable layer on top
  - Storagedriver stacks the layers on top of each other
- Copy on Write
  - If container started and file that was coming from the container changes what happens is that the file is copied to the writable layer (top layer)
  - = Copy on write from some base read-only layer to top layer
  - = top layer holds changes

#### Tags

- Format `<user_or_organization>/<repo>:tag`
- Default tag is `latest`
- Tag pointer to a specific image commit
- ImageID = only real distinguisher as multiple tags can point to the same iamge
- By default when login done pw stored in cleartext under /root/.docker/config.json
  - See: URL to configure credentialhelper

| command | comment |
|---|---|
| docker image pull <image or image:tag> | pulls an image from repo |
| docker image ls | list installed images and size |
| docker image inspect <name> | list installed images and size, metadata (ENV vars, exposed ports, CMD) |
| docker image history <name> | History of image layers, some might be file, some just metdata |
| docker image tag <srcimage:tag> <dstimage:tag> | assign one or more tags to an image |
| docker image tag <image> <image:newtag> | can also retag existing images |
| docker image tag nginx mconrad/nginx:latest | Retag existing image |
| docker login/logout | Login to registry, default is dockerhub |
| docker iamge push mconrad/nginx:latest | Upload changed layers to image registry |

### Network CLIs

| command | comment |
|---|---|
| docker network ls | Lists network drivers |
| docker network inspect <name> | Lots of info, e.g. connected containers, mtu etc. |
| docker network create --driver <driver> <name> | Create new network |
| docker network create net_app_web | by default type bridge |
| docker network connect <name> | Attach container to network |
| docker network disconnect <name> | Disconnect container from network |
| docker container run -d --name nginx --network net_app_web nginx | Start container with specific network |
| docker network connect net_app_web db | attach existing ocntainer to a certain network** |
| docker network disconnect bridge db | disconnect existing container from a network |
| ip address show | show adapter IP on host and container |
| ip link show | show adapter name on host |
| docker exec <container> ping <othercontainername> | Ping container A to B |

*docker0=bridge = default virtual network, which is NAT'ed behind host ip

**not changed but added!

```
# --network-alias = DNS RR
docker container run --network-alias elastic --network net_elastic -d --name elastic1 elasticsearch:2
docker container run --network-alias elastic --network net_elastic -d --name elastic2 elasticsearch:2

# --rm = cleanup container when finished
docker container run --rm --network net_elastic centos curl -s elastic:9200
docker container run --rm --network net_elastic centos curl -s elastic:9200
```

### General

- Docker Server also called Engine
  `docker`shows two sections (old format) and new format. New formats are under management commands.
  - docker run = all cmd style
  - docker container run = new command (cmd subcommand)
- image vs container
  - image: binary/src/libs code that make up app
    - images are pulled from registries
  - container: running instance of that image
- `Publish <host port>:<container port>` or `-p` opens local port on local machine and forwards to conainer on port xy
- `run` always starts a new container. `start` can start a stopped container
  `run` command does the following (not just start container)
  - checks local image cache
  - checks remote image repo (default dockerhub) if not found locally
  - downloads latest version (e.g. nginx:latest) if no other specified
  - creates new container based on image (adds a writable layer on top)
  - gives virtual ip on private network inside docker engine
  - opens port on host and forwards to port on container (check iptables)
  - starts container via CMD defined in Dockerfile
- Docker container just a process running on host
  - yes with restrictions (cgroups/namespaces), but can use ps -aux and see proc
  - but process (e.g. mongo db process from mongo container) can be seen on host,not running inside a VM!
- Linux Alpine ~5MB in size and focused on security (ubuntu ~72MB)
  - But e.g. has no bash installed. `docker container run -it alpine sh` instead
  - apk for package mgmt in alpine
- `docker container exec` runs an additional process, so when exiting out this additional command, the "main" container will not be stopped
- Configuration file at `/etc/docker/daemon.json`

### Docker Networks & DNS

- By default container connects to a virtual private network called "bridge"
- By default each virtual network (VN) routes via NAT on host
- `-p` not needed to expose host ports if multiple containers need to talk to each other within the host. But then new network adapter (VN) should be created
- Best practice: New Network per App
  - 1xVN: Webapp consisting of PHP/Apache Container + Mysql Container
  - 1xVN: Webapp consisting of nodejs container + mongodb container
- Container connected to same Network Adapter (e.g. bridge or docker0 by default) can talk amonst each other without restriction!
- Container can be attached to 0, one or more networks
  - `--net=host` use host ip instead of VN (to improve performance in special cases)
  - Impact on security! Attaches container directly to host interface. No more security boundary for containerization (namespace for net)
- Docker network drivers extend functionality
- Docker does NOT rely on IP (dynamically created/started/stopped)!
  - static ip possible, but anti-pattern
- Docker Deamon has a built-in DNS Server that container use by default
  - container name = what is registred in docker
  - Default bridge network does NOT have DNS built into it by default
    - Manually linking necessary via `docker container create --link nginx1: --name nginx2 nginx`
  - docker-compose spins up a new network per app automatically = comunication okey
- RR DNS built-in
  - Can have multiple containers that respond to the same DNS address
    - `--network-alias`

### Dockerfile

- Env Vars are the main ways to set keys and values
  - work everywhere, no matter what OS is the base
- Order matters of dockerfile, as each one presents a layer
  - E.g. 1 layer per RUN command = main reason why often && used to chin commands
- Very important to point logfile to stdout (12 Factor App best practice)
  - Proper way to log inside container is NOT to log into a logfile but to STDOUT/STDERR
- Built uses cached layers. BUT as soon as it hits a layer that has to be rebuilt, every layer afterwards has to be rebuilt!

- That means for example copy source code last! / towards the very end

| command | comment |
|---|---|
| docker build -f <filename> | If not named Dockerfile |
| docker image build -t mconrad/nginx:my_build . | Build image from Dockerfile |

**Dockerfile Section Overview**

| command | comment |
|---|---|
| FROM <baseimage> | Defines base image |
| LABEL <key>=<value> | Add labels to container |
| ENV <var_name> <var_value> | Defines env variables |
| RUN <cmd1> && <cmd2> | Creates a layer |
| WORKDIR /path/to/dir | = CD inside container |
| COPY <local_path> <container_path> | Copy from local to container |
| VOLUME /path/to/dir | Create a volume** |
| EXPOSE <port1> <port2> | Exposes the ports |
| CMD ["<cmd1>", "<arg1>", "<arg2>"] | Defines default cmd, can be overwritten by cli*, executed if nothing else defined |
| ENTRYPOINT ["<cmd1>", "<arg1>", "<arg2>"] | Configures container* |
| USER <user_name> | Switch to user xy for subsequent cmds |
| ADD | Ability to copy from URL & extract TAR, prefer copy if possible |
| HEALTHCHECK | Check service inside container, not just running or not |

- .dockerignore file can be used to ignore certain paths (e.g. when using COPY)
- CMD and ENTRYPOINT are preferrably defined in EXEC Form (ENTRYPOINT ["echo", "Hello World"]) vs Shell form (ENTRYPOINT echo "Hallo World"). Shell processing does NOT hapen in EXEC Form = No env variable interpretation. ** To bind mount VOLUME to specific location, we have to use CLI or docker compose. Remember Dockerfile used for building image, not how to use

```
# NOTE: this example is taken from the default Dockerfile for the official nginx Docker Hub Repo
# https://hub.docker.com/_/nginx/

FROM debian:stretch-slim
# all images must have a FROM
# usually from a minimal Linux distribution like debian or (even better) alpine
# if you truly want to start with an empty container, use FROM scratch

ENV NGINX_VERSION 1.13.6-1~stretch
ENV NJS_VERSION   1.13.6.0.1.14-1~stretch
# optional environment variable that's used in later lines and set as envvar when container is running

RUN apt-get update \
   && apt-get install --no-install-recommends --no-install-suggests -y gnupg1 \
   && \
   NGINX_GPGKEY=573BFD6B3D8FBC641079A6ABABF5BD827BD9BF62; \
   found=''; \
   for server in \
       ha.pool.sks-keyservers.net \
       hkp://keyserver.ubuntu.com:80 \
       hkp://p80.pool.sks-keyservers.net:80 \
       pgp.mit.edu \
   ; do \
       echo "Fetching GPG key $NGINX_GPGKEY from $server"; \
       apt-key adv --keyserver "$server" --keyserver-options timeout=10 --recv-keys "$NGINX_GPGKEY" && found=yes && break; \
   done; \
   test -z "$found" && echo >&2 "error: failed to fetch GPG key $NGINX_GPGKEY" && exit 1; \
   apt-get remove --purge -y gnupg1 && apt-get -y --purge autoremove && rm -rf /var/lib/apt/lists/* \
   && echo "deb http://nginx.org/packages/mainline/debian/ stretch nginx" >> /etc/apt/sources.list \
   && apt-get update \
   && apt-get install --no-install-recommends --no-install-suggests -y \
                       nginx=${NGINX_VERSION} \
                       nginx-module-xslt=${NGINX_VERSION} \
                       nginx-module-geoip=${NGINX_VERSION} \
                       nginx-module-image-filter=${NGINX_VERSION} \
                       nginx-module-njs=${NJS_VERSION} \
                       gettext-base \
   && rm -rf /var/lib/apt/lists/*
# optional commands to run at shell inside container at build time
# this one adds package repo for nginx from nginx.org and installs it

RUN ln -sf /dev/stdout /var/log/nginx/access.log \
   && ln -sf /dev/stderr /var/log/nginx/error.log
# forward request and error logs to docker log collector

EXPOSE 80 443
# expose these ports on the docker virtual network
# you still need to use -p or -P to open/forward these ports on host

CMD ["nginx", "-g", "daemon off;"]
# required: run this command when container is launched
# only one CMD allowed, so if there are multiple, last one wins
```

## Persistent Data

- Container is Immutable
  - Never change running things, if change needed -> redeploy
- Container is Ephemeral
  - Disposable
- Unique data needs to be stored somewhere
  - Volumes & Bind mounts

`Volumes: special location outside UFS`

- For example `VOLUME /var/lib/mysql`
- Tells container to create new volume and assign to that directory in the container
- Volume stays even if container deleted! Needs to be manually deleted
- For example Database outlives container lifetime
- `named volumes`: Otherwise just an ID, more userfirendly!

`Bind Mounts: link container path to host path`

- maps host file or directory into container file/directory
- can NOT be used in Dockerfile, only at runtime
- `.. run -v /Users/conrad/proj/src:/path/container`
- `.. run -v //c/users/conrad/proj/src:/path/container`
- Bind mount starts with a slash vs volume
- Local development use case
  - container detects changes and runs updates, no tools have to be installed locally

| command | comment |
|---|---|
| docker image inspect mysql | |
| docker container run -d --name mysql -e MYSQL_ALLOW_EMPTY_PASSWORD=True mysql | |
| docker container inspect mysql | See under Volumes & Mounts |
| docker volume ls | ID from docker container inspect maps |
| docker volume inspect <id> | More info on volume |
| docker container run -d ... -v /var/lib/mysql mysql | Same as VOLUME cmd in Dockerfile, ID-based volume creation |
| docker container run -d ... -v mysql-db:/var/lib/mysql mysql | Creates a named volume, docker volume ls will show name not ID |
| docker volume create | Use custom drivers and labels, docker container run uses defaults |
| docker container run -d --name nginx -p 8080:80 -v ($pwd):/usr/share/nginx/html nginx | Local development (bind mount) |
| docker container exec -it nginx bash | Run additional command,remember |
| docker container run -d --name pgdb -v psql-data:/var/lib/postgresql/data postgres:9.6.1 | Initial version of container |
| docker container logs -f pgdb | |
| docker container stop pgdb && docker container rm pgdb | delete old version |
| docker volume ls | Volume still exists after deleting container |
| docker container run -d --name pgdb -v psql-data:/var/lib/postgresql/data postgres:9.6.2 | Update container |

### Compose

- Command Line Tool & Configuration file
  - Configures relation between containers
  - "Saves" docker container run commands
  - one-liner to start/stop (dev testing = local)
  - Docker-compose still talks to Docker API!
- YAML File containing:
  - Containers
  - Volumes
  - Networks

  `docker-compose.yml`
  - has its own version (defines features in the file)
    - v1 / v2 / v.2.1 / v.3 / v3.1
  - can be used directly with docker Swarm or K8s
- `git clone https://... && docker-compose up`
- Docker-compose can build images
  - Builds with lots of vars and build args (env vars that are just available during build)
  - docker-compose up first checks if image is in cache, if not will build it
  - can trigger rebuild if running docker-compose build

| command | comment |
|---|---|
| docker-compose up | Refers to default docker-compose.yml, setup volumes, network, containers |
| docker-compose up -d && docker-compose logs | Start in background, and then show logs |
| docker-compose ps | Show containers that are runniing |
| docker-compose top | Services inside container |
| docker-compose -f somefile.yml | Other docker-compose file |
| docker-compose down | Remove containers, networks, volumes etc. |
| docker-compose build | Rebuild images |
| docker-composse up --build | Start with rebuild image |
| docker-compose down --rmi local | Removes locally built image |
| docker-compose down -v | Remove volumes |

#### docker-compose.yml Structure

```
version: '3.1'  # if no version is specified then v1 is assumed. Recommend v2 minimum

services:  # containers. same as docker run
 servicename: # a friendly name. this is also DNS name inside network
   image: # Optional if you use build:
   command: # Optional, replace the default CMD specified by the image
   environment: # Optional, same as -e in docker run
   volumes: # Optional, same as -v in docker run
 servicename2:

volumes: # Optional, same as docker volume create

networks: # Optional, same as docker network create
```

#### NGINX Example

```
version: '2'

# same as
# docker run -p 8080:80 -v $(pwd):/usr/share/nginx/html nginx/latest

services:
 ngiii:
   image: nginx:latest
   volumes:
           - .:/usr/share/nginx/html
   ports:
           - '8080:80'
```

- Example to mark file as read-only in container
- And configure nginx proxy to httpd (using DNS name web in nginx.conf)

```
version: '3'

services:
 proxy:
   image: nginx:1.13 # this will use the latest version of 1.13.x
   ports:
           - '80:80' # expose 80 on host and sent to 80 in container
   volumes:
           - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
 web:
   image: httpd  # this will use httpd:latest

# nginx.conf
server {
   listen 80;
   location / {
       proxy_pass         http://web;
       proxy_redirect     off;
       proxy_set_header   Host $host;
       proxy_set_header   X-Real-IP $remote_addr;
       proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
       proxy_set_header   X-Forwarded-Host $server_name;

   }
}
```

**Bigger example**

- `depends_on` option would help compose to understand relationship = which container to start first
- environment vars can be specified in 2 ways:
  - `VAR: value` no dash
  - `- VAR=value` with dash

```
version: '2'

services:

 wordpress:
   image: wordpress
   ports:
           - 8080:80
   environment:
     WORDPRESS_DB_HOST: mysql
     WORDPRESS_DB_NAME: wordpress
     WORDPRESS_DB_USER: example
     WORDPRESS_DB_PASSWORD: examplePW
   volumes:
           - ./wordpress-data:/var/www/html
   depends_on:
           - db

 db:
   image: mariadb
   environment:
     MYSQL_ROOT_PASSWORD: examplerootPW
     MYSQL_DATABASE: wordpress
     MYSQL_USER: example
     MYSQL_PASSWORD: examplePW
   volumes:
           - mysql-data:/var/lib/mysql

volumes:
 mysql-data:
```

**Build example**

- `context` specifies directory (=Where to look for Dockerfile)
- Use case: local development with volume mounted for static pages into container
- Very simple dockerfile
- Uses same nginx.conf as above (NGINX Example)

```
version: '2'

# builds nginx.conf into image
# uses sample HTML static site from https://startbootstrap.com/themes/agency/

services:
 proxy:
   build:
     context: .
     dockerfile: nginx.Dockerfile
   #Optional -- image: nginx-custom
   ports:
           - '80:80'
 web:
   image: httpd
   volumes:
           - ./html:/usr/local/apache2/htdocs/

FROM nginx:1.13

COPY nginx.conf /etc/nginx/conf.d/default.conf
```

### Swarm

- Containers bring new Problems!
- How to scale out/in/up/down?
- How to automate container lifecycle?
- How to ensure they are recreated if they fail?
- How to replace conatainers without downtime (blue/green deploy)?
  - = Rolling update
- How to control/track where containers get started?
- How to create cross-node virtual networks?
- How to ensure only trusted servers run containers?
- How can secrets, passwords, keys be stored and only visible to the correct container?
- Swarm = Clustering/Orchestration solution built into docker
  - bring together different OS/nodes into a single managable unit
  - Swarm Mode added in 1.12 (2016) (SwarmKit)
  - Enhanced in 1.13 via Stacks and Secrets (2017)
- Not enabled by default
  - `docker swarm` && `docker node` && `docker service` && `docker stack` && `docker secret`
- Concepts
  - Manager nodes have `raft database`
  - Worker nodes, e.g. linux server running dockerd
    - `docker node ls` fails: Workers are not priviliged to control swarm, only managers
  - Manager can also be worker
  - Single `service` can have multiple `tasks`, and each task will launch a `container`
  - `docker service` replaces `docker run` command to spin up a service (1 or more containers)

  `docker swarm init` starts swarm mode
  - PKI and security automation (Root Signing Certs, certs issued to 1st manager, join tokens)
  - Raft DB created to store root CA, configs and secrets
    - encrypted on disk (1.13+)
    - = no need anymore for another key value store (like redis) on the system to hold e.g. secrets
    - replicates logs amongst managers via mutual TLS in "control plane"
- Swarm Concepts
  - Overlay Multi-Host Network
  - Routing Mesh

| command | comment |
|---|---|
| docker info \| grep Swarm | Swarm inactive by default |
| docker swarm init | Initialize single node swarm |
| docker swarm join --token <token> <ip>:2377 | Add node to swarm |
| docker swarm leave | Take node out from swarm |
| docker node ls | List nodes and swarm manager |
| docker service <cmd> | replaces docker run (single host) to swarm |
| docker service create alpine ping 8.8.8.8 | Creates service |
| docker service ps <id_name> | shows on which node it runs |
| docker service scale <service>=<numreplicas> | Example to scale out a service |
| docker service scale reverent_rhodes=2 | Example to scale out to 2 |
| docker service update <name> --replicas 3 | Scale up option 2 |
| docker container ls | still works |
| docker container inspect --format '{{json .Config.Labels }}' <name;name> \| jq | Shows the labels associated with the container = info for swarm |
| docker container rm -f <name>.1.<id> | Swarm will detect failure and start a new container |
| docker service rm <name> | remove service (shut containers etc.) |
| docker swarm init --advertise-addr x.x.x.x | Advertise manager on reachable addr |
| docker node update --role manager node2 | Change role of a worker to manager |
| docker swarm join-token manager | Show join token for node to become manager when joining swarm |
| docker service create --replicas 3 alpine ping 8.8.8.8 | Gets distributed in swarm across nodes |
| docker service ps <servicename> | Shows distribution of containers across swarm |
| docker node ps <nodename> | Shows services running on s speciic node |
| --- if using virtualbox --- | --- |
| docker-machine create node1 | Swarm Setup - Virtualbox + mini LInux VM |

| | |
|---|---|
| docker-machine create node2 | Swarm Setup - Virtualbox + mini LInux VM |
| docker-machine ssh node1 | SSH into node |
| docker-machine env node1 && eval(docker-machine env node1) | SHows env vars & points docker cli to specific node |
| docker service logs <servicename> | Serivce Logs from all nodes |
| docker container logs <container-name> | See logs from a container |

Other options for Swarm setup - Play with Docker - https://play-with-docker.com - 4h wiped - https://training.play-with-docker.com - Digital Ocean - Create 3 Droplets (=Servers)

**Swarm Feature - Overlay Network**

- Creates a swarm wide bridge network
- containers on different hosts behave like on a single VLAN
- Optional IPSec (AES) encryption on the network
  - tunnels between each 2 hosts
- Service can be added to 0, 1 or 2+ overlay networks
  - e.g. front-end and backend

| command | comment |
|---|---|
| docker network create --driver overlay <name> | New overlay network across swarm |
| docker network ls | List networks |
| docker service create --name psql --network drupal_nw -e POSTGRES_PASSWORD=pgdb postgres | |
| docker service create --name drupal --network drupa_nw -p 8080:80 drupal | |
| docker service ls | List the service |
| docker service ps psql | List a part (process) of the service |
| docker container logs psql | Check logs |
| watch docker service ls | Linux run cmd every 2 sec |
| docker serivce inspect drupal | Check for virtualIP (on overlay network) |

**Swarm Feature - Routing Mesh**

- Routes Ingress packets for a service to proper Task
  - E.g. even if Frontend runs on NodeX, it is reachable via all Node IP Addresses on that particual port (e.g. 8080)
- Spans ALL nodes in swarm
- Uses `IPVS` from Linux Kernel
- Load balances (and listens) swarm services across their tasks
  - Container-to-container: In overlay network via VIP
    - E.g. Frontend-to-backend communicates via VIP if backend has multiple DBs
    - No load balancer needed, swarm does that
  - External traffic coming to published ports (all nodes listen)
    - Container might fail and be put onto different node
  - Load balancer actually built-in to the `overlay` network driver
- Not DNS-RR (which would pose a problem to real load-balancing due to caching)
  - VIP in front
- Features Notes
  - 17.03: Stateless load balancing (session cookies & hit specific container for a session)
    - Nginx or HA Proxy container that sit in front
    - Docker enterprise wich has a built-in L4 web proxy
  - LB is at OSI Layer 4 (TCP), not Layer 5 (DNS)

| command | comment |
|---|---|
| docker service create --name search --replicas 3 -p 9200:9200 elasticsearch:2 | |
| curl localhost 9200 | Query Elasticsearch |
| docker service ps search | Replicas distributed |

**Service Example**

- Example app using Swarm
  - Note: `-v` not available in swarm, have to use `--mount type=volume,source=xy,target=yz`

```
docker network create --driver overlay frontend
docker network create --driver overlay backend
docker service create --name db --network backend --mount type=volume,source=db-data,target=/var/lib/postgresql/data -e POSTGRES_HOST_AUTH_METHOD=trust postgres:9.4
docker service create --name vote --network frontend -p 8080:80 --replicas 2 bretfisher/examplevotingapp_vote
docker service create --name redis --network frontend redis:3.2
docker service create --name worker --network frontend --network backend bretfisher/examplevotingapp_worker
docker service create --name result --network backend -p 5001:80 bretfisher/examplevotingapp_result

docker service ls
docker service ps vote
docker service ps worker
docker service logs <service-name>
docker container logs <container-name>
```

**Swarm Stacks**

- = something like Production grade compose
  - docker-compose used for local development
  - service commands used bash scripts, now possibility to use compose file to define/declar service description
- Version 1.13 stacks accepts `docker-compose.yml` files
- Includes: Networks, services, volumes, secrets

  `build:` not supported in compse file,
  - Build should happen somehwere else, e.g. CI and push to repo
  - Stack pulls images from repository

`deploy:` dict instead for defining swarm-specific stuff
    - Replicas, fail over, rolling updates
- Can use same docker-compose file
    - Compose ignores `deploy:` --> Local machine
    - Swarm ignores `build:` --> Production machine
- `docker-compose` cli not needed on swarm
- Labels put in to control metadata
- Version 3 or higher needed in order to use Stacks!
- Update procedure
    - `deploy` command acts also as update, just change the yml file and deploy again
    - yml file = SoT = Source of Truth
- Secrets
    - Built into since Version 1.13
        - Swarm Raft DB Encrypted on disk, encrypted in transit
        - Only stored on disk of manager node
        - Keys provided to worker node via control plane (encrypted TLS channel)
        - Secrets 1st stored in swarm, then assigned to services
        - Only container in assigned service can see them (other container no access)
            - docker engine keep it in memory and only allows access for certain containers
            - looks like file on harddrive to container. Uses `in-memory fs` or `ram fs`
            - `/run/secrets/<secret-name>`
            - `/run/secrets/<secret-alias>`
        - Local development can use file-based secrets, but this is unsecure!
            - Faked so can use same docker-compose file locally.
        - Secrets depend on swarm = only available in swarm
    - What is a secret?
        - Username/pw
        - TLS certs and keys
        - SSH keys
        - Twitter api key etc.
    - Support generic strings or binary up to 500Kb in size

| command | comment |
|---|---|
| docker stack deploy -c <yml> name | Uses compose files. Instead of docker service create |
| docker stack ls | List stuff |
| docker stack ps <stackname> | list node state etc |
| docker stack services <stackname> | list ports replicas etc |
| docker stack rm <name> | remove a stack |
| docker network ls | shows networks that were created for stack |
| docker stack deploy -c <yml> name | No update on stack cmd, just change yml and run again |
| docker secret create <name_in_swarm_db> <file_or_cmd> | Create secret in Swarm DB** |
| echo "pw" \| docker secret create <name_in_swarm_db> - | Create from CLI,* |
| docker secret ls | Show secrets in Swarm DB |
| docker secret inspect <name> | Does NOT show pw, but can when created |
| docker service create --name psql --secret psql_user --secret psql_pw -e POSTGRES_PASSWORD_FILE=/run/secrets/psql_pw -e POSTGRES_USER_FILE=/run/secrets/psql_user postgres | Example using secrets |
| docker exec -it <container_name> bash | Login to container and check `/run/secrets/...` |
| ls /run/secrets && df | Tmpfs mount created and files there |
| docker service update --secret-rm psql_pw | Redeploy container |

- *** Dash (-) = read from stdin
- ** Drawback: Stored on HD in host.
- **** Drawback: Visibiliy in history
- could use remote api on cli on local machine

```
version: "3"
services:
 redis:
   image: redis:alpine
   networks:
           - frontend
   deploy:
     replicas: 1
     update_config:
       parallelism: 2
       delay: 10s
     restart_policy:
       condition: on-failure
 db:
   image: postgres:9.4
   volumes:
           - db-data:/var/lib/postgresql/data
   networks:
           - backend
   environment:
           - POSTGRES_HOST_AUTH_METHOD=trust
   deploy:
     placement:
       constraints: [node.role == manager]
 vote:
   image: mconrad/examplevotingapp_vote
```

```
    ports:
            - 5000:80
    networks:
            - frontend
    depends_on:
            - redis
    deploy:
      replicas: 2
      update_config:
        parallelism: 2
      restart_policy:
        condition: on-failure
  result:
    image: mconrad/examplevotingapp_result
    ports:
            - 5001:80
    networks:
            - backend
    depends_on:
            - db
    deploy:
      replicas: 1
      update_config:
        parallelism: 2
        delay: 10s
      restart_policy:
        condition: on-failure

  worker:
    image: mconrad/examplevotingapp_worker
    networks:
            - frontend
            - backend
    depends_on:
            - db
            - redis
    deploy:
      mode: replicated
      replicas: 1
      labels: [APP=VOTING]
      restart_policy:
        condition: on-failure
        delay: 10s
        max_attempts: 3
        window: 120s
      placement:
        constraints: [node.role == manager]

  visualizer:
    image: mconrad/visualizer
    ports:
            - 8080:8080
    stop_grace_period: 1m30s
    networks:
            - frontend
    volumes:
            - /var/run/docker.sock:/var/run/docker.sock
    deploy:
      placement:
        constraints: [node.role == manager]

networks:
  frontend:
  backend:

volumes:
  db-data:
```

#### Inheritance, Secrets and Local Development

For secrets the yaml would be extended as follows. Requires `version 3.1`: - File means that secret is located in a file, not secure (testing only) - External means the secret is already in the raft DB, however it was added - Can be used in production (e.g. add via CLI and remove history) - Real World might have 3 Docker-compose files - 1 for Local Development `docker-compose up` with secret file - 1 for CI Testing `docker-compose up` with secret file - 1 for Prod `docker stack deploy` with secret external - Inheritance - `docker-compose.yml` - defines base for all images - `docker-compose.override.yml` - automatically executed when using local docker-compose up command! - `docker-compose.test.yml` and `docker-compose.prod.yml` - need `-f` and `docker-compose config` to combine multiple files - `extends:` option to override compose files from base - See `swarm-stack-3` examples!

| command | comment |
|---|---|
| docker-compose up -d | Automatically takes docker-compose.yml base and docker-compose.override.yml and combines them |
| docker-compose -d -f docker-compose.yml -f docker-compose.test.yml up | Combine multiple dockerfiles, base file always needs to come first |
| docker-compose -f docker-compose.yml -f docker-compose.prod.yml config > combined.yml | Combine two files and output the resulting config |
| docker stack deploy -c <yml> name | Uses compose files. Instead of docker service create |

```
version: "3.1"

services:
  psql:
    image: postgres
    # Defined secrets are only assigned to this container
    # Longer cmd availble for non-root user to specify exact permissions
```

```
   secrets:
           - psql_user
           - psql_password
   environment:
     POSTGRES_PASSWORD_FILE: /run/secrets/psql_password
     POSTGRES_USER_FILE: /run/secrets/psql_user

secrets:
 psql_user:
   file: ./psql_user.txt
 psql_password:
   file: ./psql_password.txt

# Production
# secrets:
#  psql-pw:
#    external: true
```

#### Swarm service updates

- Rolling updates of tasks/containers by default
- Usually replaces container (with new ones)
- Rollback and healthcheck options
- Scale and rollback most often used = quick format
  - `docker service scale web=4` and `docker service rollback web`
- Stack `deploy` on a already running stack =update

| command | comment |
|---|---|
| docker service update --image appxy:1.5.1 <svc_name> | Update an image |
| docker service update --env-add VAR1=prod --publish-rm 8080 <svc_name> | Remove port, add env var |
| docker service update --publish-rm 8080 --publish-add 9090 web | change port |
| docker service scale web=8 api=2 | Change number replicas |
| watch docker service ps web | See rolling update |
| docker service rollback web | Rollback |
| docker stack deploy -c file.yml <stack_name> | Stack cmd will issue necessary api calls to match definition |

#### Labels & Constraints

- Control container placement with labels & constraints.
- Examples:
  - VPN should run on nodes in a DMZ
  - Calculations stuff shoul run on nodes with an SSD
  - Different availability zones/subnets
  - Windows on windows worker nodes

### HEALTHCHECK

- Feature added in 1.12, before only check if container still running
- Works in Dockerfile, Compose YAML, docker run, Swarm

  Will `exec` the command INSIDE the container
  - For example `curl localhost`
  - Works even without exposed ports

  Expects `exit 0=OK` or `exit 1=ERROR`
  - Needs exit 1, not other exit codes
  - Often uses `|| false` to get around other codes or e.g. http query
- Container states:
  - `starting` (30sec) or `healthy` (runs every 30sec by default) or `unhealthy`
- Defaults
  - Interval: 30sec by default
  - Timeout: 30sec by default
  - start period: 0sec by default (sth that takes e.g long to start)
  - retries: 3 by default
- Not a monitoring replacement
- Docker run does NOT take action
- Swarm stack&services DO take action if healthcheck fails

| command | comment |
|---|---|
| docker container ls | Shows health check (healthy or unhealthy) |
| docker container inspect | shows last 5 health check results |
| docker container inspect -f "{{ json .State.Health }}" <name;name> | Example Query output |

```
docker run \
--health-cmd="curl -f localhost:9200/_cluster/health || false" \
--health-interval=5s \
--health-retries=3 \
--health-timeout=2s \
--health-start-period=15s \
elasticsearch:2

FROM NGINX:1.13
# Option 1
HEALTHCHECK curl -f http://localhost/ || false
# Option 2
```

```
HEALTHCHECK --timeout=2s --interval=3s --retries=3 \
 CMD curl -f http://localhost/ || exit 1

FROM postgres
# Option 1
HEALTHCHECK curl -f http://localhost/ || false
# Option 2
HEALTHCHECK --timeout=3s --interval=3s --retries=3 \
 CMD pg_isready -U postgres_user || exit 1

version: "3.4"
services:
 web:
   image: nginx
   healthcheck:
     test: ["CMD", "curl", "-f", "http://localhost"]
     interval: 1m30s
     timeout: 10s
     retries: 3
     # requires 3.4 version
     start_period: 1m
```

### Own Registry

- Public image available called `registry`
- UI Catalog accessible via `https://name:5000/v2/_catalog/`
- Swarm needs to push/pull from central repository (local build might not be an option)
  - Therefore local registry might be a good option
  - Routing mesh steers towards correct container
- Private registries need to store data somehwere! volumes!
- Docker hub option to trigger webhook to start other build stuff like travis, jenkins
- https communication only for registries (except localhost)
- Public registries
  - Docker Hub
  - Quay.io
  - Docker Registry
  - Docker EE (self-hosted)
  - Gitlab container registry (self-hosted)
  - Quay Enterprise (self-hosted)
  - Docker Enterprise Edition (DTR) = Docker trusted registry
  - See: Awesome docker

| command | comment |
|---|---|
| docker image run -d -p 5000:5000 --name registry -v ($pwd)/registry-data:/var/lib/registry registry | Runs a private registry |
| docker image pull hello-world | Pull an image from Dockerhub |
| docker image tag hello-world 127.0.0.1:5000/hello-world | Retag image, important needs IP and port of registry in there |
| docker image push 127.0.0.1:5000/hello-world | Push to local registry |
| tree registry-data | Check data in the registry |
| docker image remove hello-world | Remove from loccal chache |
| docker iamge rm 127.0.0.1:5000/hello-world | Remove from local chache |
| docker pull 127.0.0.1:5000/hello-world | Get from local registry |

### Cleanup

| command | comment |
|---|---|
| docker image prune | Cleanup unused images |
| docker system prune [-a] | Cleanup everything (-a = all that have no container associated) |
| docker system df | See free space |
| docker volume prune | Delete unused volumes |

### Windows Containers

- What it is? Native Windows .exe binaries running in Docker containers on a Windows kernel

### Random Stuff

- Tools that might be missing inside the container
  - `ip address show && ip link show` - provided by iproute2 pkt (ubuntu)
  - `ps aux` - provided by procps pkt
- Use `apt install apt-file && apt-file search --regexp 'bin/ip$'` to identify packet
- Use `$(pwd)` on Unix-like systems (mac, linux) and `${pwd}` on Powershell and %cd% on CMD in windows
- Iptables:

| command | comment |
|---|---|
| iptables -S [chain] | List Rules for a specific chain |
| sudo ufw enable | Enable uncomplicated fw (iptables manager) on ubuntu |

### References

- See: Docker Run vs Entrypoint
- See: Cloud Native Foundation Charta
- See: Free Docker Swarm Playground
- See: Free Docker Labs

- See: BretFisher Udemy Course
- Most of the notes come from this course, as well as images/screenshots. If you find this helpful, support Bret and buy his courses! I do not get money for this comment.